

ERPChat Unified System Document: Schema Dictionary & Business Logic Standard

The absolute source of truth combining detailed database entity mappings, calculation criteria, financial standards, and query guardrails to ensure mistake-free RAG indexing.

PART 1: DATA DICTIONARY & SCHEMA MAPS

Each block below represents a completely self-contained text chunk for embedding retrieval models, preserving table summaries alongside explicit data field schemas and structural relationships.

Table 1: products

Description: Central catalog storing all core item metadata, SKU specifications, baseline pricing configurations, and taxation multiplier defaults.

Column Name	Data Type	Key Type	Description / Constraints
product_id	int4	Primary Key	Unique auto-incrementing identifier for each product record.
sku	varchar	Unique	Stock Keeping Unit; unique internal/vendor alphanumeric identifier code string.
product_name	varchar	-	Commercial descriptive name of the item.
category	varchar	-	Organizational classification group for inventory routing and metrics reports.
cost_price	numeric	-	Internal production or baseline manufacturing/acquisition cost profile per unit.
unit_price	numeric	-	Current active standard outgoing wholesale or retail sell price.
tax_rate	numeric	-	Applicable standard tax rate multiplier percentage assigned to the product.
is_discontinued	bool	-	Status flag signifying if the catalog item is permanently retired.

Relationships & Foreign Keys: Referenced by `inventory.product_id` (One-to-Many), `order_items.product_id` (One-to-Many), and `stock_movements.product_id` (One-to-Many).

Table 2: inventory

Description: Tracks real-time physical stock metrics, safety reorder thresholds, and transactional reservation allocations across distinct warehouses.

Column Name	Data Type	Key Type	Description / Constraints
inventory_id	int4	Primary Key	Unique sequence identity tracking key for the inventory record entry.
product_id	int4	Foreign Key	References products.product_id to attach stock counts to catalog definitions.
warehouse_name	varchar	-	Descriptive location name or distinctive code of the active warehouse node.
stock_on_hand	int4	-	Total balance of items physically present inside the designated warehouse site.
reorder_level	int4	-	Minimum physical safety balance count triggering replenishment automated pipelines.
allocated_stock	int4	-	Items reserved for confirmed orders that are not yet packed or shipped.

Relationships & Foreign Keys: Many-to-One connection back to products.product_id .

Table 3: stock_movements

Description: Comprehensive chronological audit ledger tracking every physical stock mutation, warehouse transfer, baseline receipt, or manual adjustment.

Column Name	Data Type	Key Type	Description / Constraints
movement_id	int4	Primary Key	Unique tracking sequential primary key for the ledger event entry.
product_id	int4	Foreign Key	References products.product_id indicating the modified product profile.
warehouse_name	varchar	-	The specific warehouse facility context where the stock delta was logged.
movement_type	varchar	-	The mutation state classification (e.g. 'INBOUND', 'OUTBOUND', 'ADJUSTMENT').
quantity	int4	-	The numerical item balance modifier size (positive or negative transaction counts).
reference_id	int4	-	Optional foreign key trace reference to an underlying ticket or origin order document.
movement_timestamp	timestampz	-	Exact timestamp when the ledger change was formally committed to disk.

Relationships & Foreign Keys: Many-to-One structural mapping straight to products.product_id .

Table 4: customers

Description: Commercial buyer master profile data registry, housing credit boundaries, current accounting balances, and regional location metadata.

Column Name	Data Type	Key Type	Description / Constraints
<code>customer_id</code>	int4	Primary Key	Unique baseline auto-increment profile index identifier string.
<code>company_name</code>	varchar	-	The officially registered business name of the corporate account buyer.
<code>contact_name</code>	varchar	-	Primary human point-of-contact handling account communications.
<code>email</code>	varchar	Unique	Primary billing contact email address (must remain unique across rows).
<code>phone</code>	varchar	-	Contact numbers on file for delivery routing and payment notifications.
<code>credit_limit</code>	numeric	-	Maximum numerical balance threshold permitted for credit-based purchase orders.
<code>outstanding_balance</code>	numeric	-	Current unpaid dollar/rupee financial value balances aggregated on account.
<code>city</code>	varchar	-	Locality geographic indicator parameter for accounts invoicing.
<code>state</code>	varchar	-	State or regional administrative province location label.
<code>country</code>	varchar	-	National boundary identification marker.
<code>is_active</code>	bool	-	State flag representing operational eligibility for purchasing.
<code>created_at</code>	timestampz	-	Creation profile record log timestamp.

Relationships & Foreign Keys: Referenced explicitly by `sales_orders.customer_id` (One-to-Many relations mapping).

Table 5: sales_orders

Description: Structural header tracking data for commercial sales transactions, final net pricing aggregations, status loops, and payment condition records.

Column Name	Data Type	Key Type	Description / Constraints
order_id	int4	Primary Key	Unique transactional header sequential identification document index.
customer_id	int4	Foreign Key	References customers.customer_id mapping ownership of the sale document.
order_date	timestamptz	-	Standard formal registration timestamp of order confirmation.
status	varchar	-	System lifecycle state string (e.g. 'Draft', 'Pending', 'Shipped', 'Paid', 'Cancelled').
subtotal	numeric	-	Aggregated monetary cost profile sum of line totals BEFORE taxes.
tax_amount	numeric	-	Total quantitative taxation calculations computed across the order subtotal.
total_amount	numeric	-	Net final payload invoice cost due for financial settlement (Subtotal + Tax).
payment_terms	varchar	-	Contractual timeframe configurations (e.g. 'NET30', 'COD', 'Net60').
sales_rep_id	int4	Foreign Key	References employees.employee_id defining the staff agent closing the account.

Relationships & Foreign Keys: Many-to-One to **customers** , Many-to-One to **employees** . Referenced by **order_items.order_id** (One-to-Many cascade link).

Table 6: order_items

Description: Granular structural line tiers itemizing individual products, specific quantities bought, discounts, and custom row line calculations.

Column Name	Data Type	Key Type	Description / Constraints
item_id	int4	Primary Key	Unique tracking sequential line index entry row identifier.
order_id	int4	Foreign Key	References parent invoice document via sales_orders.order_id .
product_id	int4	Foreign Key	References catalog profile specifications via products.product_id .
quantity	int4	-	The item unit counts requested and purchased under this line.
unit_price	numeric	-	Fixed transactional snapshot price calculated/applied at time of booking.
discount_percent	numeric	-	Deduction percentage rate modifier applied directly to row items.
line_total	numeric	-	Final row net cost total factoring in price, count, and discounts.

Relationships & Foreign Keys: Many-to-One structural join paths straight into both sales_orders and products .

Table 7: employees

Description: Human Resource internal workspace matrix tracking profiles, identity variables, financial salary items, and active structural nodes.

Column Name	Data Type	Key Type	Description / Constraints
<code>employee_id</code>	int4	Primary Key	Unique serial workforce indicator code assigned to the employee.
<code>first_name</code>	varchar	-	Worker given name.
<code>last_name</code>	varchar	-	Worker family legal surname tracking record.
<code>email</code>	varchar	Unique	Unique corporate enterprise mail address string.
<code>hire_date</code>	date	-	Standard formal initiation calendar date tracking employment launch.
<code>dept_id</code>	int4	Foreign Key	References <code>departments.dept_id</code> assigning team alignment.
<code>job_title</code>	varchar	-	Work title or standard operational designation role name.
<code>salary</code>	numeric	-	Base payroll compensation parameters.
<code>is_active</code>	bool	-	Flag determining current active staff status.

Relationships & Foreign Keys: Maps Many-to-One into `departments` . Referenced by `departments.manager_id` (Self-Referencing Manager nodes), `sales_orders.sales_rep_id` , and `payslips.employee_id` .

Table 8: departments

Description: Organizational company functional divisions connected directly to specific executive management nodes.

Column Name	Data Type	Key Type	Description / Constraints
<code>dept_id</code>	int4	Primary Key	Unique business team indicator identification code.
<code>dept_name</code>	varchar	Unique	Unique textual definition title of the corporate vertical group.
<code>manager_id</code>	int4	Foreign Key	References <code>employees.employee_id</code> assigning a functional leader node.

Relationships & Foreign Keys: Many-to-One to `employees` for manager tracking; referenced by `employees.dept_id` for functional hierarchy layout.

Table 9: accounts_chart

Description: Master ledger definition tracking asset types, liabilities configurations, equity balances, and system currency configurations.

Column Name	Data Type	Key Type	Description / Constraints
<code>account_code</code>	varchar	Primary Key	Unique bookkeeping account alphanumeric index indicator code.
<code>account_name</code>	varchar	-	Corporate financial account title.
<code>account_type</code>	varchar	-	Classification label (e.g. 'ASSET', 'LIABILITY', 'REVENUE', 'EXPENSE').
<code>currency</code>	varchar	-	Standard base ISO currency code structure (e.g. 'INR', 'USD', 'EUR').

Relationships & Foreign Keys: Referenced by `general_ledger.account_code` (One-to-Many booking paths).

Table 10: general_ledger

Description: Core accounting system logging tables capturing absolute immutable double-entry double ledger transaction entries.

Column Name	Data Type	Key Type	Description / Constraints
<code>entry_id</code>	uuid	Primary Key	Globally unique identifier string assigned per accounting modification row.
<code>account_code</code>	varchar	Foreign Key	References <code>accounts_chart.account_code</code> .
<code>entry_date</code>	timestampz	-	Formal calendar date of tracking accounting recognition parameters.
<code>debit</code>	numeric	-	Monetary debit mutation adjustment amount parameters.
<code>credit</code>	numeric	-	Monetary credit mutation adjustment amount parameters.
<code>reference_order_id</code>	int4	Foreign Key	Optional tracking trace joining back to <code>sales_orders.order_id</code> .
<code>description</code>	text	-	Text annotations documenting transaction context elements.

Relationships & Foreign Keys: Many-to-One to `accounts_chart`, Many-to-One tracing to `sales_orders` via `reference_order_id`.

Table 11: Infrastructure Blocks (payslips & audit_logs)

- Description:** Internal ecosystem utility layers mapping recurring employee transactions and automated system data audit histories.
- **payslips:** Contains `payslip_id` (int4, PK), `employee_id` (int4, FK pointing to `employees.employee_id`), periodic boundaries, gross salaries, deductions tracking, and final payout timestamp items.
 - **audit_logs:** System health database tracking user login activities, modified row payloads, previous state tracking variables, and action context logging.

PART 2: CONSOLIDATED SYSTEM BUSINESS RULES

The rules below represent the combined calculation logic parameters, default financial behaviors, and specific numeric parsing models mapped across the system architecture.

Section 1: Revenue & Financial Calculation Metrics

Rule 1.1: Line-Level Value Extraction Formula

Individual row computations must evaluate dynamically via standard decimal formatting rules, applying baseline pricing snapshots, volume metrics, and explicit percent discounts:

$line_total = quantity \times unit_price \times (1 - (discount_percent / 100))$

SQL Reference Implementation:

```
SELECT item_id,
       (quantity * unit_price * (1.0 - (discount_percent / 100.0))) AS calculated_line_total
FROM order_items;
```

LLM Guardrail: Never reference product baseline pricing configurations from the `products` master catalog table when evaluating historical performance. Always sum or retrieve values directly through `order_items.unit_price` to protect calculation integrity from shifting master catalog edits.

Rule 1.2: Realized Billed Revenue Standard & Status Filter Boundaries

Revenue reflects **finalized billed sales records only**. Non-realized transaction staging parameters are handled via strict filter boundaries:

- **Standard Operational Omissions:** By default, all revenue summaries must explicitly exclude non-realized data entries via: `WHERE sales_orders.status NOT IN ('Draft', 'Cancelled')`.
- **"All Orders" Execution Clause:** Include draft/cancelled entities *only* if the user question explicitly targets unfinalized orders or explicitly asks for "all orders". When compiling this path context, append a clear structural comment statement emphasizing that this displays an alternate, non-default interpretation.

Rule 1.3: Revenue Metric Splitting (Tax-Inclusion Handling)

Financial revenue inquiries must evaluate tax parameters according to precise textual triggers:

- **Default Total Revenue (Tax-Inclusive View):** Calculated on order-level figures:
`SUM(sales_orders.total_amount)`. This represents the standard business financial framework.
- **Pre-tax Revenue View:** Evaluated using `SUM(sales_orders.subtotal)`. This calculation rule is engaged *only and exclusively* if the query explicitly demands or asks for "pre-tax subtotal" figures.
- **Product-Tier Revenue Rankings:** Rank performance itemizations by grouping and sorting line metrics:
`SUM(order_items.line_total) GROUP BY product_id`. Do not mix product-level performance rankings with order-level header totals to prevent calculation inflation errors.

Rule 1.4: Date Boundaries, Time Context, & Financial Assumptions

- **Time Bounds Alignment:** Filter periodic revenue summaries (Yearly, Quarterly, Monthly tracks) strictly through boundaries defined against the transaction date field: `sales_orders.order_date`. (Note: Current system baseline year context tracking is 2026).
- **KPI Accounting Integrity:** Financial KPI data meaning must be preserved rigorously even when tracking tables or interfaces truncate response rows for short list display views.

Default Financial Assumptions Framework: Whenever user query intent introduces ambiguity, prefer conservative corporate reporting defaults: use realized/finalized sales only, filter out Draft/Cancelled entries, enforce tax inclusions within total fields, and show an explicit note stating the underlying structural assumptions to the interface.

Section 2: Credit limits, Profiles & Currency (INR Conventions)

Rule 2.1: Indian Numbering (Lakh Unit Conversions)

Credit evaluations and user threshold queries operate relative to Indian National Rupee (INR) rules. Convert language-based number expressions into raw numbers within SQL comparison parameters:

- **1 Lakh** = 100,000 (e.g. `100000`)
- **5 Lakhs** = 500,000 (e.g. `500000`)

Rule 2.2: Credit Limits vs Outstanding Account Balances

Do not confuse or substitute line thresholds for debt metrics inside filter expressions:

- **High-Credit Queries:** Isolate targets by filtering customer metrics straight through `customers.credit_limit` using the numerical threshold parsed from the text.
- **Outstanding Balances:** Address ongoing customer liability or debt inquiries strictly through `customers.outstanding_balance` fields. Do not swap this out for credit limits.
- **Risk Breaches Protocol:** An account falls out of clear guidelines when: `outstanding_balance > credit_limit`

```
-- Filter active high-risk balance violations
SELECT customer_id, company_name, credit_limit, outstanding_balance
FROM customers
WHERE is_active = true AND outstanding_balance > credit_limit;
```

Section 3: Inventory Architecture, SKUs & Warehouse Scope

Rule 3.1: SKU Lookups & Warehouse Identifications

- **SKU Query Mapping:** Match product stock identifiers by filtering directly against `products.sku` text strings, mapping warehouse inventory status details through `product_id` primary/foreign key table joins.
- **Facility Enumerations KPI:** To isolate the total volume of active warehouse nodes, apply explicit distinct checks: `COUNT(DISTINCT inventory.warehouse_name)` . Use aggregates for KPI questions instead of returning large, raw, unsummarized row sets.

Rule 3.2: Real Sellable Available Quantities & High-Velocity Metrics

- **Warehouse Available Balance formula:** Evaluate stock assets per node exactly via:

available_stock = stock_on_hand - allocated_stock

- **Low Stock replenished triggers:** Flags trigger when available metrics match the rule:

available_quantity ≤ reorder_level

- **Top-Selling Fast-Moving velocity parameters:** Identify current high-performing items by ranking rows according to current calendar year sales volumes: **SUM(order_items.quantity) AS units_sold** sorted in descending sequence.

```
-- Top Velocity Inventory Analysis Standard (Current Year = 2026)
SELECT oi.product_id, SUM(oi.quantity) AS units_sold
FROM order_items oi
JOIN sales_orders so ON oi.order_id = so.order_id
WHERE EXTRACT(YEAR FROM so.order_date) = 2026
      AND so.status NOT IN ('Draft', 'Cancelled')
GROUP BY oi.product_id
ORDER BY units_sold DESC;
```

Section 4: Cross-Domain Restrictions & Ledger Security

Rule 4.1: Cross-Domain Boundary Isolation (Multi-Step Guardrail)

Absolute Join Restriction Guardrail: Customer profiles (Credit limits, balances, context) and Warehouse logistics (SKU lookups, inventory levels, allocations) represent completely separate functional targets. **Do not join customers directly to inventory** tables unless the user explicitly requests a complex, cross-domain multi-step calculation metric.

Rule 4.2: Double-Entry Balancing & System Integrity Check

The general ledger dictates that global or transaction-grouped debits must explicitly balance with corresponding credit adjustments down to zero variance:

$$\Sigma(\text{debit}) = \Sigma(\text{credit})$$

If a validation or health check query evaluates out of balance, use the inequality equation constraint `SUM(debit) != SUM(credit)` combined with a `HAVING` operator to isolate broken records.

```
-- Ledger error balancing check
SELECT reference_order_id, SUM(debit) as total_debit, SUM(credit) as total_credit
FROM general_ledger
GROUP BY reference_order_id
HAVING SUM(debit) != SUM(credit);
```

Rule 4.3: Corporate Hierarchy Mapping (Self-Referencing Joins)

The system human resource architecture loops cyclically through management rows. Map managers and supervisors using targeted foreign key criteria structures:

- **Isolate employee direct managers:** Join `employees e JOIN departments d ON e.dept_id = d.dept_id JOIN employees m ON d.manager_id = m.employee_id`.
- **Department Head listings:** Direct join via `departments d JOIN employees e ON d.manager_id = e.employee_id`.